

▼ Módulo 5: Bibliotecas matemáticas

Certifique-se de que você está dentro da pasta da aula antes de executar o comando abaixo. O comando criará automaticamente as subpastas `cpp/src`, `cpp/bin` e `cpp/lib` dentro dela. Caso tenha executado anteriormente, não é necessário reexecutar.

```
!mkdir -p cpp/src cpp/bin cpp/lib
```

A estrutura gerada será a seguinte.

```
NOME_DA_AULA/
├── cpp/
│   ├── src/
│   ├── bin/
│   └── lib/
```

Pasta	Conteúdo	Extensões
<code>src</code>	Arquivos fonte com a lógica do programa	<code>.cpp</code>
<code>bin</code>	Executáveis gerados pelo compilador	sem extensão
<code>lib</code>	Cabeçalhos e bibliotecas externas	<code>.h</code> <code>.hpp</code>

Nota sobre os comandos do Jupyter Notebook

Você vai notar que os exemplos desta aula utilizam alguns comandos especiais, como `%%writefile`, `!g++` e `!./...`. É importante entender que **esses prefixos não fazem parte do C++**, O `%%writefile` e o `!` são apenas uma "ponte" entre o Jupyter e o terminal do sistema. **O fluxo real é o mesmo que você vê nos vídeos:** escrever o código, compilar com g++ e executar o binário. — eles são recursos do próprio Jupyter Notebook que nos permitem escrever e executar código C++ diretamente por aqui.

- `%%writefile cpp/src/02_exec_02.cpp` salva o conteúdo da célula como um arquivo `.cpp` no disco.
- `!g++ cpp/src/02_exec_02.cpp -o cpp/bin/02_exec_02` compila o arquivo usando o g++, exatamente como você faria no terminal.
- `!./cpp/bin/02_exec_02` executa o binário gerado.

Disponibilizamos os exemplos dessa forma para que você consiga acompanhar e rodar tudo diretamente na aula, sem precisar sair do notebook, de modo online e interativo. Mas recomendamos que, sempre que possível, **pratique também pelo terminal**, como demonstrado nos vídeos — é assim que você vai trabalhar no dia a dia com C++.

▼ 5.1 Introdução à biblioteca `<cmath>`

Em C++, os operadores aritméticos nativos (`+`, `-`, `*`, `/`, `%`) cobrem as operações básicas. Porém, quando precisamos de funções matemáticas mais avançadas — como calcular potências, raízes quadradas, logaritmos ou funções trigonométricas — recorremos à biblioteca `<cmath>`. Ela faz parte da Standard Library do C++ e fornece um conjunto abrangente de funções prontas para uso, organizadas nas categorias a seguir.

 **Dica:** Todas as funções de `<cmath>` estão no namespace `std`. Se você não usar `using namespace std;`, precisará chamá-las com o prefixo `std::`, como `std::sqrt(25)`.

Categoria	Função	Descrição
Operações básicas	<code>abs(x)</code>	Retorna o valor absoluto de x (versão inteira e ponto flutuante)
	<code>fabs(x)</code>	Retorna o valor absoluto de x (exclusiva para ponto flutuante)
	<code>fmod(x, y)</code>	Resto da divisão de x por y (ponto flutuante)
	<code>remainder(x, y)</code>	Resto IEEE da divisão de x por y
	<code>fmax(x, y)</code>	Retorna o maior entre x e y
	<code>fmin(x, y)</code>	Retorna o menor entre x e y
	<code>fdim(x, y)</code>	Diferença positiva entre x e y (retorna x - y se x > y, senão 0)
	<code>fma(x, y, z)</code>	Multiplicação-adição fundida: calcula x * y + z em uma única operação
Arredondamento	<code>ceil(x)</code>	Arredonda para cima (menor inteiro ≥ x)
	<code>floor(x)</code>	Arredonda para baixo (maior inteiro ≤ x)
	<code>round(x)</code>	Arredonda para o inteiro mais próximo
	<code>lround(x)</code>	Igual a <code>round</code> , mas retorna <code>long</code>
	<code>llround(x)</code>	Igual a <code>round</code> , mas retorna <code>long long</code>
	<code>trunc(x)</code>	Trunca a parte decimal (arredonda em direção a zero)
	<code>nearbyint(x)</code>	Arredonda usando o modo de arredondamento atual
	<code>rint(x)</code>	Semelhante a <code>nearbyint</code> , mas pode sinalizar exceção de inexatidão

Categoria	Função	Descrição
Exponenciais e potências	<code>exp(x)</code>	Retorna e elevado à potência x (e^x)
	<code>exp2(x)</code>	Retorna 2 elevado à potência x (2^x)
	<code>expm1(x)</code>	Retorna $e^x - 1$ (mais preciso para x próximo de zero)
	<code>pow(base, exp)</code>	Retorna base elevada à potência exp
	<code>sqrt(x)</code>	Raiz quadrada de x
	<code>cbrt(x)</code>	Raiz cúbica de x
	<code>hypot(x, y)</code>	Hipotenusa: calcula $\sqrt{(x^2 + y^2)}$ sem risco de overflow
Logaritmos	<code>log(x)</code>	Logaritmo natural (base e) de x
	<code>log2(x)</code>	Logaritmo de base 2 de x
	<code>log10(x)</code>	Logaritmo de base 10 de x
	<code>log1p(x)</code>	Retorna $\log(1 + x)$ (mais preciso para x próximo de zero)
	<code>logb(x)</code>	Extrai o expoente de x (base FLT_RADIX)
	<code>ilogb(x)</code>	Extrai o expoente de x como inteiro
Trigonômicas	<code>sin(x)</code>	Seno de x (x em radianos)
	<code>cos(x)</code>	Cosseno de x (x em radianos)
	<code>tan(x)</code>	Tangente de x (x em radianos)
	<code>asin(x)</code>	Arco-seno (inversa do seno) — retorna radianos
	<code>acos(x)</code>	Arco-cosseno (inversa do cosseno) — retorna radianos
	<code>atan(x)</code>	Arco-tangente (inversa da tangente) — retorna radianos
	<code>atan2(y, x)</code>	Arco-tangente de y/x , considerando o quadrante correto
Hiperbólicas	<code>sinh(x)</code>	Seno hiperbólico de x
	<code>cosh(x)</code>	Cosseno hiperbólico de x
	<code>tanh(x)</code>	Tangente hiperbólica de x
	<code>asinh(x)</code>	Inversa do seno hiperbólico
	<code>acosh(x)</code>	Inversa do cosseno hiperbólico
	<code>atanh(x)</code>	Inversa da tangente hiperbólica
	<code>atanh2(y, x)</code>	Arco-tangente de y/x , considerando o quadrante correto
Ponto flutuante	<code>isnan(x)</code>	Verifica se x é NaN (Not a Number)
	<code>isinf(x)</code>	Verifica se x é infinito
	<code>isfinite(x)</code>	Verifica se x é um valor finito
	<code>isnormal(x)</code>	Verifica se x é um número normal (não subnormal, não infinito, não NaN)
	<code>signbit(x)</code>	Verifica se o bit de sinal de x está ativado
	<code>copysign(x, y)</code>	Retorna x com o sinal de y
	<code>nan(tag)</code>	Gera um valor NaN silencioso
Especiais (C++17)	<code>tgamma(x)</code>	Função gama: $\Gamma(x)$
	<code>lgamma(x)</code>	Logaritmo natural do valor absoluto da função gama
	<code>erf(x)</code>	Função erro
	<code>erfc(x)</code>	Função erro complementar ($1 - \text{erf}(x)$)
	<code>beta(x, y)</code>	Função beta: $B(x, y)$

Atenção: Todas essas funções retornam `double`, não `int`. Se você precisar de um inteiro, faça o cast explícito:

Para mais funções, acesse a documentação oficial: cppreference.com — [Common mathematical functions](http://common-mathematical-functions.com)

5.2 Exemplos práticos

5.2.1 Arredondamento: `floor()`, `ceil()` e `round()`

As funções de arredondamento permitem controlar como valores fracionários são convertidos para inteiros. A função `floor()` sempre arredonda para baixo (em direção a $-\infty$), enquanto `ceil()` sempre arredonda para cima (em direção a $+\infty$). Já `round()` arredonda para o inteiro mais próximo.

```
%writefile cpp/src/05_exec_01.cpp
#include <iostream>
using namespace std;

int main() {

    double weight {7.7};

    // floor – arredonda para baixo
    std::cout << "floor(7.7) = " << std::floor(weight) << std::endl;
    // Saída: 7

    // ceil – arredonda para cima
    std::cout << "ceil(7.7) = " << std::ceil(weight) << std::endl;
    // Saída: 8

    // round – arredonda para o mais próximo
    std::cout << "round(7.7) = " << std::round(weight) << std::endl;
```

```
// Saída: 8  
  
return 0;  
}
```

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/05_exec_01.cpp -o cpp/bin/05_exec_01
```

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/05_exec_01`, ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

```
!./cpp/bin/05_exec_01
```

5.2.2. Valor absoluto: `abs()`

A função `abs()` retorna o valor absoluto de um número, ou seja, remove o sinal negativo. É especialmente útil ao trabalhar com diferenças, distâncias ou valores financeiros onde o sinal não importa.

```
%writefile cpp/src/05_exec_02.cpp  
#include <iostream>  
using namespace std;  
  
int main() {  
  
    double savings {-5000};  
    double weight {7.7};  
  
    std::cout << "abs(7.7) = " << std::abs(weight) << std::endl;  
    // Saída: 7.7  
  
    std::cout << "abs(-5000) = " << std::abs(savings) << std::endl;  
    // Saída: 5000  
  
    return 0;  
}
```

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/05_exec_02.cpp -o cpp/bin/05_exec_02
```

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/05_exec_02`, ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

```
!./cpp/bin/05_exec_02
```

5.2.3. Exponenciais e potências: `exp()` e `pow()`

A função `exp(x)` calcula e elevado à potência x , onde $e \approx 2.71828$ (constante de Euler). Já `pow(base, expoente)` calcula qualquer potência, elevando a base ao expoente fornecido.

```
%writefile cpp/src/05_exec_03.cpp  
#include <iostream>  
using namespace std;  
  
int main() {  
  
    // exp: calcula e^x (e ≈ 2.71828)  
    double exponencial = std::exp(10);  
    std::cout << "exp(10) = " << exponencial << std::endl;  
    // Saída: 22026.5  
  
    // pow: calcula base^expoente  
    std::cout << "3^4 = " << std::pow(3, 4) << std::endl;  
    // Saída: 81  
  
    std::cout << "9^3 = " << std::pow(9, 3) << std::endl;  
    // Saída: 729  
}
```

```
    return 0;
}
```

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/05_exec_03.cpp -o cpp/bin/05_exec_03
```

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/05_exec_03`, ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

```
!./cpp/bin/05_exec_03
```

5.2.4. Logaritmos: `log()` e `log10()`

Logaritmos são a operação inversa da exponenciação. A função `log(x)` calcula o logaritmo natural (base e) de x , enquanto `log10(x)` calcula o logaritmo de base 10. Em termos práticos: se `exp(y) = x`, então `log(x) = y`.

```
%%writefile cpp/src/05_exec_04.cpp
#include <iostream>
using namespace std;

int main() {

    // exp: calcula e^x (e ≈ 2.71828)
    double exponencial = std::exp(10);
    std::cout << "exp(10) = " << exponencial << std::endl;
    // Saída: 22026.5

    // pow: calcula base^expoente
    std::cout << "3^4 = " << std::pow(3, 4) << std::endl;
    // Saída: 81

    std::cout << "9^3 = " << std::pow(9, 3) << std::endl;
    // Saída: 729

    return 0;
}
```

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/05_exec_04.cpp -o cpp/bin/05_exec_04
```

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/05_exec_04`, ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

```
!./cpp/bin/05_exec_04
```

5.2.5. Funções trigonométricas: `sin()`, `cos()` e `tan()`

As funções trigonométricas `sin()`, `cos()` e `tan()` recebem o ângulo em **radianos** (não em graus). Para converter graus em radianos, use a fórmula: `radianos = graus ×  $\pi$  / 180`.

```
%%writefile cpp/src/05_exec_05.cpp
#include <iostream>
using namespace std;

int main() {

    // Seno, cosseno e tangente (argumento em radianos)
    double angulo = 3.14159 / 4; // 45 graus

    std::cout << "sin(45°) = " << std::sin(angulo) << std::endl;
    // Saída: ≈ 0.7071

    std::cout << "cos(45°) = " << std::cos(angulo) << std::endl;
    // Saída: ≈ 0.7071
}
```

```
std::cout << "tan(45°) = " << std::tan(angulo) << std::endl;
// Saída: ≈ 1.0

return 0;
}
```

Para compilar o programa acima, rode o comando abaixo:

```
!g++ cpp/src/05_exec_05.cpp -o cpp/bin/05_exec_05
```

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/05_exec_05`, ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

```
!./cpp/bin/05_exec_05
```

No exemplo acima, o valor de π foi digitado manualmente (`3.14159`), o que funciona, mas não oferece a melhor precisão. A biblioteca `<cmath>` disponibiliza constantes matemáticas que resolvem isso, como `M_PI`, `M_E` e `M_SQRT2`. No GCC e Clang, elas já ficam disponíveis automaticamente ao incluir `<cmath>`.

As constantes mais comuns são:

Constante	Valor	Descrição
<code>M_PI</code>	3.14159265358979...	π
<code>M_PI_2</code>	1.57079632679489...	$\pi / 2$
<code>M_PI_4</code>	0.78539816339744...	$\pi / 4$
<code>M_E</code>	2.71828182845904...	e (base do logaritmo natural)
<code>M_SQRT2</code>	1.41421356237309...	$\sqrt{2}$
<code>M_LN2</code>	0.69314718055994...	$\ln(2)$
<code>M_LN10</code>	2.30258509299404...	$\ln(10)$
<code>M_LOG2E</code>	1.44269504088896...	$\log_2(e)$
<code>M_LOG10E</code>	0.43429448190325...	$\log_{10}(e)$

As constantes `M_*` são extensões POSIX, não fazem parte do padrão ISO C++. A partir do **C++20**, o padrão oferece constantes oficiais no header `<numbers>`:

```
# include <numbers>

...

std::numbers::pi;    // 3.14159265358979...
std::numbers::e;     // 2.71828182845904...
std::numbers::sqrt2; // 1.41421356237309...
```

Refatorando o exemplo anterior com `<numbers>`:

```
%%writefile cpp/src/05_exec_06.cpp
#include <iostream>
#include <numbers> // C++20
#include <cmath>
using namespace std;

int main() {

    // Seno, cosseno e tangente (argumento em radianos)
    const double PI {std::numbers::pi};

    std::cout << "sin(45°) = " << std::sin(PI/4) << std::endl;
    // Saída: ≈ 0.7071

    std::cout << "cos(45°) = " << std::cos(PI/4) << std::endl;
    // Saída: ≈ 0.7071

    std::cout << "tan(45°) = " << std::tan(PI/4) << std::endl;
    // Saída: ≈ 1.0

    return 0;
}
```

Para compilar o programa acima, é necessário informar a flag `-std=c++20`, pois o header `<numbers>` exige essa versão do padrão:

```
!g++ -std=c++20 cpp/src/05_exec_06.cpp -o cpp/bin/05_exec_06
```

Para executar o programa, você tem duas opções: rode o binário diretamente pelo terminal (nativo ou integrado do VSCode) navegando até a pasta da aula e executando `./cpp/bin/05_exec_06`, ou utilize a célula abaixo no próprio Jupyter Notebook para facilitar.

```
!./cpp/bin/05_exec_06
```